

PARALEL PROGRAMLAMA İLE POLİGON İÇERİSİNDE NOKTA BULMA

Paralel Programlama Dersi — Teknik Proje Raporu

Tuğba Nur Ayık - 22360859035

1. GİRİŞ

Günümüzde geometrik hesaplamalar, bilgisayar grafikleri, coğrafi bilgi sistemleri (GIS), robotik ve oyun geliştirme gibi birçok alanda yaygın olarak kullanılmaktadır. Bu hesaplamalar arasında önemli problemlerden biri, verilen bir noktanın belirli bir poligonun içerisinde bulunup bulunmadığının belirlenmesidir. Bu problem literatürde Point-In-Polygon (PIP) problemi olarak bilinmektedir.

Büyük veri kümeleri ile çalışılan uygulamalarda poligonun kenar sayısının artması, hesaplama maliyetini de doğrudan artırmaktadır. Geleneksel sıralı (sequential) algoritmalar tüm kenarları tek bir işlem akışı içerisinde değerlendirdiğinden, büyük boyutlu verilerde performans kayıpları ortaya çıkabilmektedir.

Bu projede Point-In-Polygon problemi paralel programlama teknikleri kullanılarak çözülmüştür. Çözümde Java programlama dili ve çok iş parçacıklı (multithreaded) yapı kullanılmıştır. Amaç, poligonun kenarlarını farklı iş parçacıkları arasında paylaştırarak işlem süresini azaltmak ve paralel programlamanın performans üzerindeki etkisini incelemektir.

2. KULLANILAN YÖNTEM

Çözümde yaygın olarak kullanılan Ray Casting (Işın İzleme) algoritmasından yararlanılmıştır.

Ray Casting algoritmasında test edilmek istenen noktadan sağa doğru sonsuz uzunlukta hayali bir ışın gönderilir. Bu ışının poligonun kenarları ile yaptığı kesişim sayısı hesaplanır.

Algoritmanın temel mantığı aşağıdaki kurala dayanmaktadır:

- Kesişim sayısı tek ise nokta poligonun içerisinde.
- Kesişim sayısı çift ise nokta poligonun dışındadır.

Bu yaklaşımın tercih edilme nedeni uygulamasının kolay olması ve konveks (convex) veya konkav (concave) tüm poligon tiplerinde çalışabilmesidir.

Algoritmanın zaman karmaşıklığı:

$O(n)$

olarak ifade edilmektedir. Burada n poligonun kenar sayısını göstermektedir.

3. PARALEL ÇÖZÜM

Projede Java'nın Thread yapısı kullanılarak paralel bir çözüm geliştirilmiştir.

İlk olarak poligonun kenarları eşit büyüklükte parçalara ayrılmıştır. Daha sonra her parça farklı bir iş parçacığına atanmıştır. Sistemde toplam dört adet iş parçacığı kullanılmıştır.

Görev dağılımı aşağıdaki şekilde gerçekleştirilmiştir:

Thread-1 → İlk %25 kenar

Thread-2 → İkinci %25 kenar

Thread-3 → Üçüncü %25 kenar

Thread-4 → Son %25 kenar

Her iş parçacığı kendisine ait kenarlar üzerinde Ray Casting algoritmasını bağımsız olarak çalıştırmıştır. Elde edilen kesişim sayıları AtomicInteger veri yapısı kullanılarak güvenli bir şekilde ortak sonuç değişkeninde birleştirilmiştir.

İş parçacıklarının tamamlanması için join() metodu kullanılmış ve tüm işlemler sona erdikten sonra toplam kesişim sayısına göre noktanın poligonun içerisinde veya dışında olduğu belirlenmiştir.

Bu yöntem sayesinde işlem yükü birden fazla işlemci çekirdeğine dağıtılmış ve paralel çalışma gerçekleştirilmiştir.

4. KULLANILAN FONKSİYONLAR VE SINIFLAR

Projede aşağıdaki Java sınıf ve fonksiyonlarından yararlanılmıştır.

run()

Her iş parçacığının gerçekleştireceği işlemler bu fonksiyon içerisinde tanımlanmıştır. Thread çalıştırıldığında otomatik olarak çağrılmaktadır.

start()

Yeni bir iş parçacığı oluşturur ve çalışmasını başlatır.

join()

Ana programın ilgili iş parçacığının tamamlanmasını beklemesini sağlar.

AtomicInteger

İş parçacıkları tarafından hesaplanan kesişim sayılarının güvenli şekilde toplanmasını sağlamaktadır. Böylece veri yarışları (race condition) önlenmiştir.

addAndGet()

AtomicInteger üzerinde atomik toplama işlemi gerçekleştirmektedir.

nanoTime()

Programın çalışma süresini nanosaniye cinsinden ölçmek amacıyla kullanılmıştır.

Point Sınıfı

Poligon köşelerinin ve test noktasının x ve y koordinatlarını tutmak amacıyla geliştirilmiştir.

5. DENEYSEL SONUÇLAR

Program farklı büyüklüklerdeki poligonlar üzerinde test edilmiştir. Her testte sıralı çözüm ve paralel çözüm süreleri ölçülmüş, ardından hızlanma katsayısı hesaplanmıştır.

Hızlanma katsayısı aşağıdaki formül kullanılarak hesaplanmıştır:

$$Speedup = \frac{T_{sequential}}{T_{parallel}}$$

Burada:

- $T_{sequential}$: Sıralı çözüm süresi
- $T_{parallel}$: Paralel çözüm süresi

olarak tanımlanmaktadır.

Deney 1 – 10.000.000 Nokta

10000000 nokta:

```
Sequential: true
Parallel: true
Sequential Time(ns): 44076800
Parallel Time(ns): 35883300
Speedup: 1.2283374160124627
```

Deney 2 – 50.000.000 Nokta

```
Sequential: true
Parallel: true
Sequential Time(ns): 305521500
Parallel Time(ns): 83086500
Speedup: 3.677149717463126
```

Deney 3 – 100.000.000 Nokta

Bu deney sırasında sistem belleğinin yetersiz kalması nedeniyle aşağıdaki hata ile karşılaşmıştır:

```
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space  
at ParallelPointInPolygon.main(ParallelPointInPolygon.java:74)
```

Bu durum, poligonun tüm noktalarının bellekte tutulmasının yüksek miktarda RAM gerektirmesinden kaynaklanmaktadır. 100 milyon adet nokta için gerekli bellek miktarı mevcut sistem kaynaklarını aşmıştır.

Bu nedenle üçüncü deney teorik olarak değerlendirilmiş ve sistem kaynaklarının izin verdiği ölçüde daha küçük veri kümeleri kullanılmıştır.

6. DEĞERLENDİRME

Elde edilen sonuçlar incelendiğinde paralel programlamanın büyük veri kümelerinde önemli performans avantajları sağlayabildiği görülmüştür. Özellikle poligonun kenar sayısı arttıkça iş yükü işlemci çekirdekleri arasında daha etkin şekilde dağıtılabilmektedir.

Küçük veri kümelerinde iş parçacığı oluşturma ve senkronizasyon maliyetleri nedeniyle paralel çözüm beklenen performansı göstermeyebilmekte, hatta bazı durumlarda sıralı çözüme göre daha yavaş çalışabilmektedir. Ancak veri boyutu büyüdükçe bu maliyetlerin etkisi azalmakta ve paralel yaklaşımın avantajı ortaya çıkmaktadır.

DeneySEL sonuçlar paralel programlamanın hesaplama yoğun uygulamalarda performansı artırmak için etkili bir yöntem olduğunu göstermiştir.

7. SONUÇ

Bu projede bir noktanın verilen poligonun içerisinde bulunup bulunmadığını belirleyen Point-In-Polygon problemi paralel programlama yaklaşımı ile çözülmüştür. Çözümde Ray Casting algoritması kullanılmış ve işlem yükü dört farklı iş parçacığına dağıtılmıştır.

Gerçekleştirilen deneyler sonucunda paralel çözümün büyük veri kümelerinde daha verimli çalıştığı gözlemlenmiştir. Ayrıca çalışma sırasında karşılaşılan bellek kısıtları değerlendirilmiş ve yüksek hacimli veriler üzerinde paralel algoritmaların yalnızca işlemci gücü değil bellek kullanımı açısından da optimize edilmesi gerektiği görülmüştür.

Sonuç olarak, paralel programlama tekniklerinin geometrik problemlerin çözümünde performansı artırdığı ve büyük ölçekli uygulamalar için önemli avantajlar sunduğu belirlenmiştir.